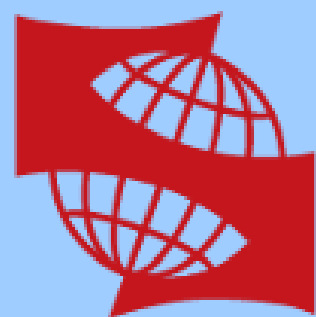




SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)

Constituent of Symbiosis International (Deemed University), Pune

(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of India)

Re-Accredited by NAAC with 'A++' Grade

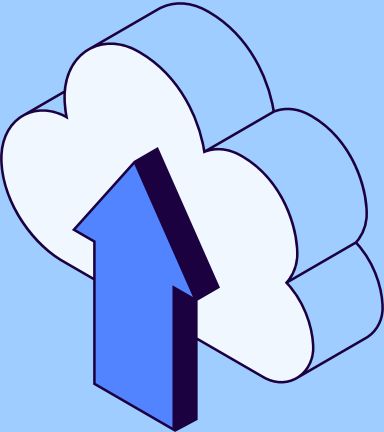
॥वसुधैव कुटुम्बकम्॥

DevOps Theory CA - 2

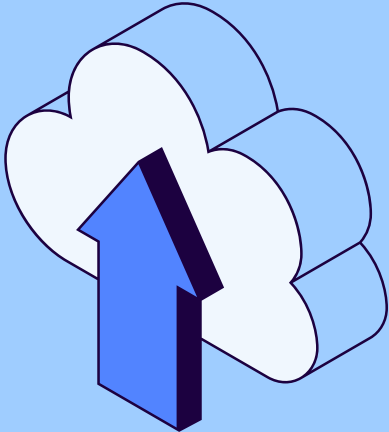
Local Rate Limiter Microservice

Architecture & Deployment Report

Presented by: Uttkarsh Ruparel
PRN: 23070122227

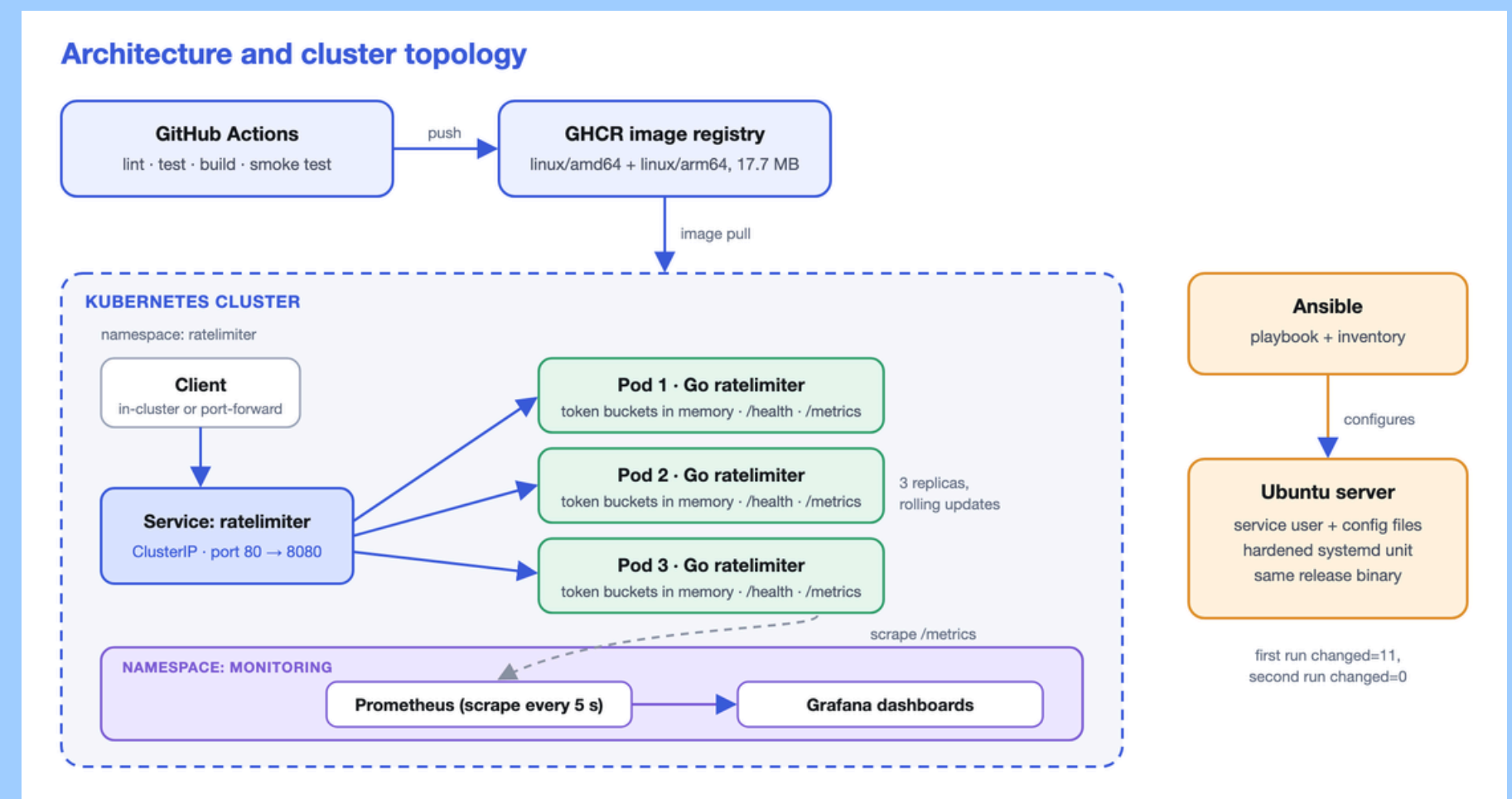


Architecture



A rate-limiting service built as a small, stateless-by-design Go binary: one immutable image, configured only by environment, deployed identically on Kubernetes and on a plain server.

- Layered, minimal design. The limiter core, the HTTP layer and the config are separate packages. The core knows nothing about HTTP, which keeps it small and easy to test.
- Horizontally scalable by design. Each pod is self-contained and keeps its token buckets in memory, so replicas scale out with no shared database. The trade-off is that limits are per pod, a deliberate choice of simplicity and speed over global consistency.
- 3. Build once, run anywhere. One static binary becomes one immutable image, configured only through environment variables. The same artifact runs on Kubernetes and on a plain Ubuntu server under Ansible.



Automated CI/CD with GitHub Actions

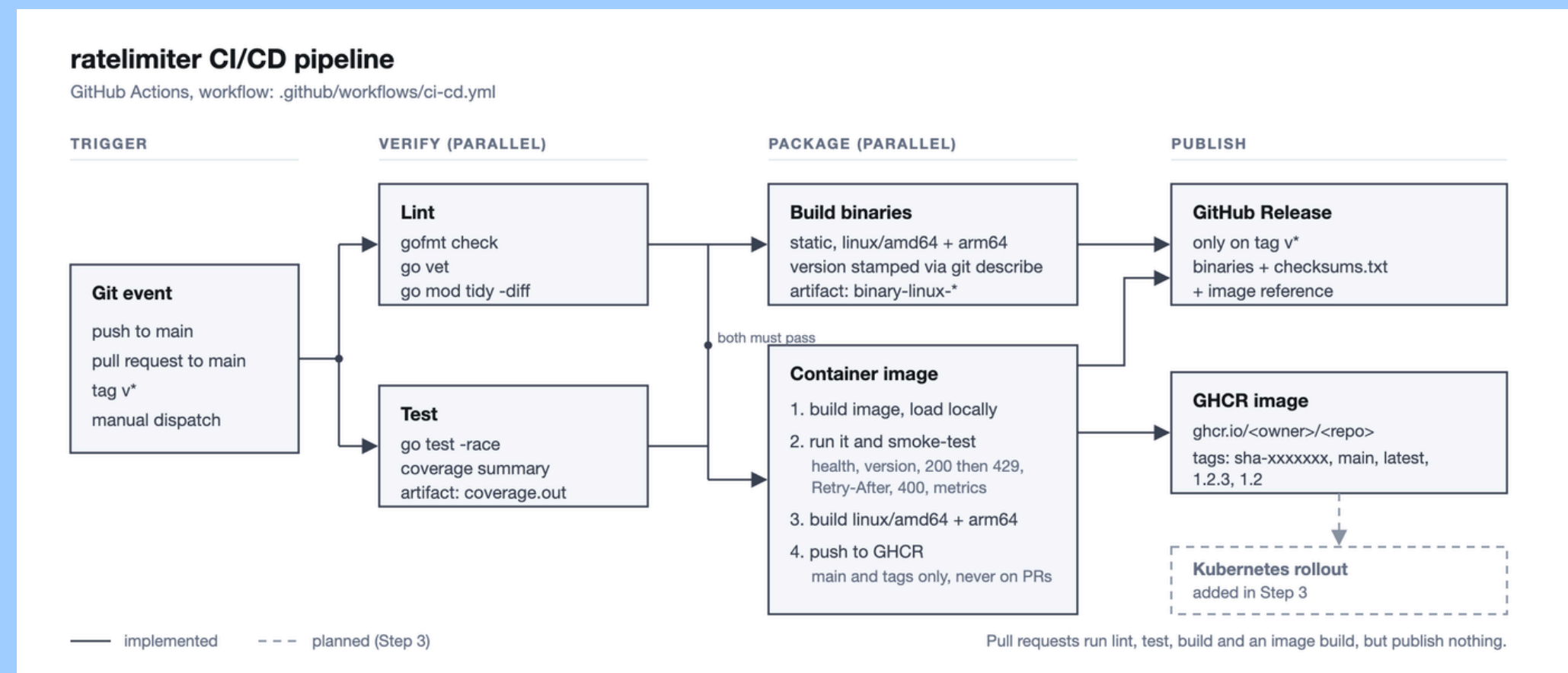
Pipeline stages

- **Lint:** gofmt, go vet and a dependency tidy check.
- **Test:** unit and HTTP tests with the race detector, plus a coverage summary.
- **Build:** static Linux binaries for amd64 and arm64, with the version stamped in.
- **Container image:** build, run it, smoke-test it, then push a multi-arch image to GHCR.
- **Release:** a v* tag publishes a GitHub Release with binaries and checksums.

Every push is linted, tested, built and smoke-tested, and only a passing build on main or a release tag publishes an image.

Components

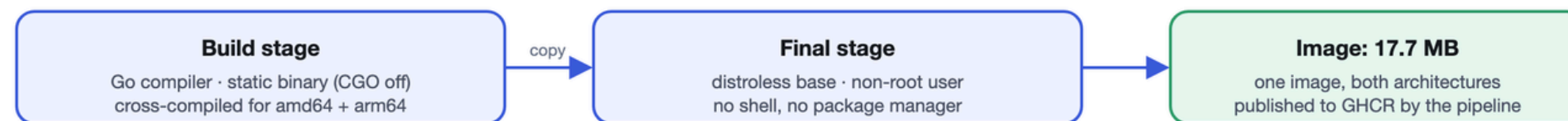
- **Workflow file:** ci-cd.yml defines the whole pipeline as code.
- **Triggers:** push to main, pull requests, and v* tags.
- **Go toolchain:** gofmt, go vet and go test.
- **Docker Buildx:** builds the multi-arch image (amd64 and arm64).
- **Smoke-test script:** smoke.sh runs 10 checks against the real container.
- **GHCR:** GitHub Container Registry stores the published images.
- **GitHub Release:** holds the binaries and checksums for tagged versions.



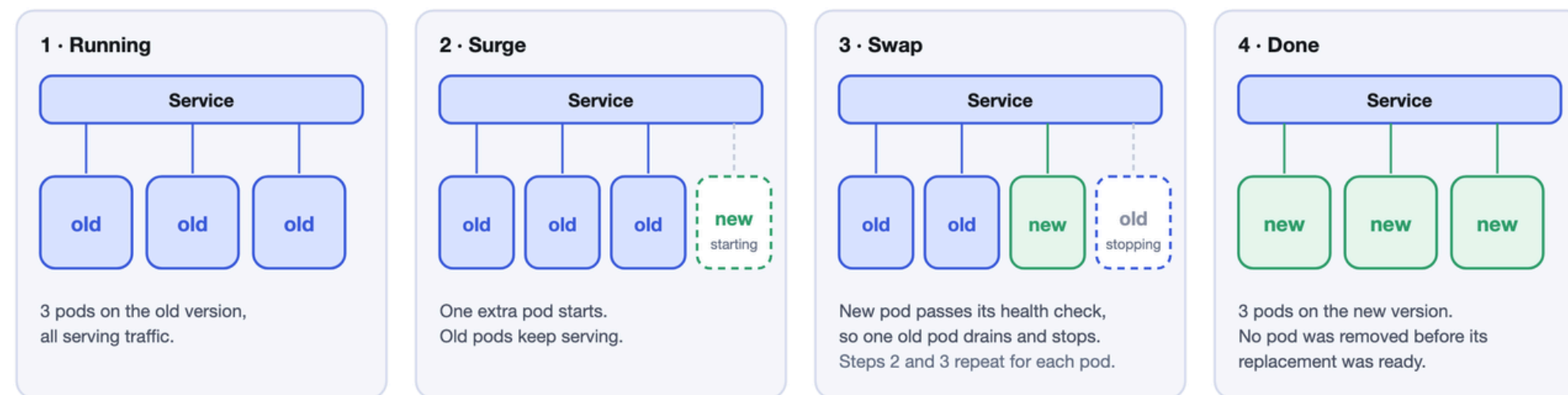
Containerization and Orchestration with Docker and Kubernetes

Docker image and Kubernetes rolling update

DOCKER: TWO-STAGE BUILD



KUBERNETES: ROLLING UPDATE, 3 REPLICAS



maxSurge 1 · maxUnavailable 0

Capacity never drops below 3 pods during an update.

Measured: 21 s rollout, 0 failed requests

A bad release is held back and rolled back with one command.

Docker

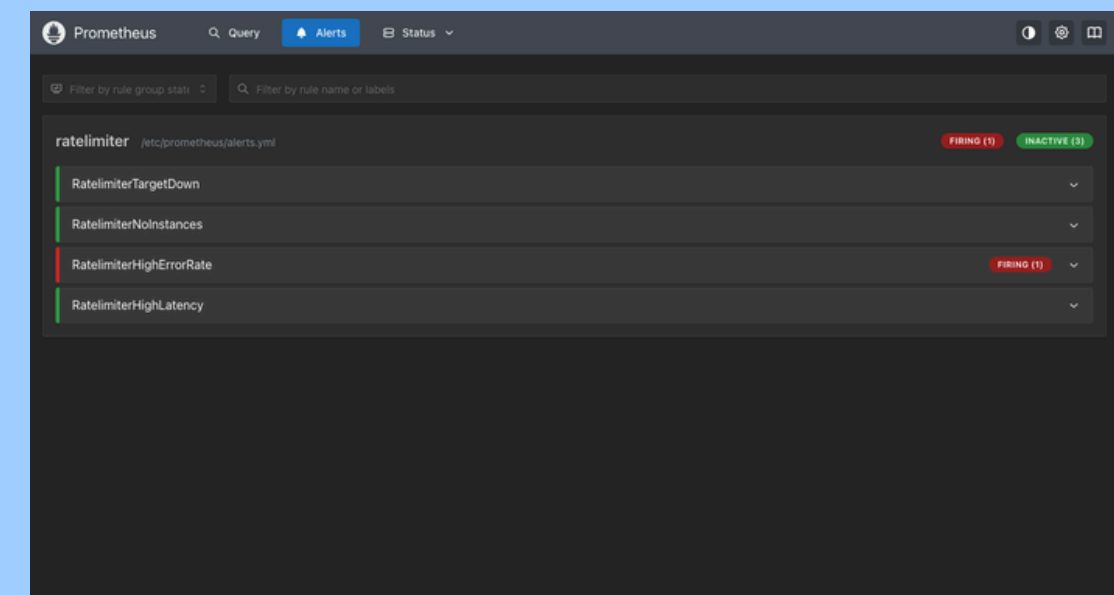
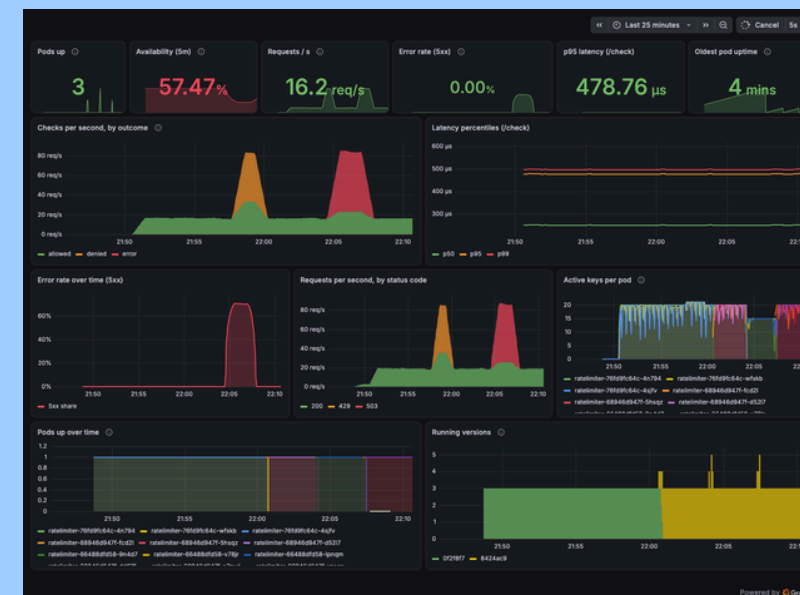
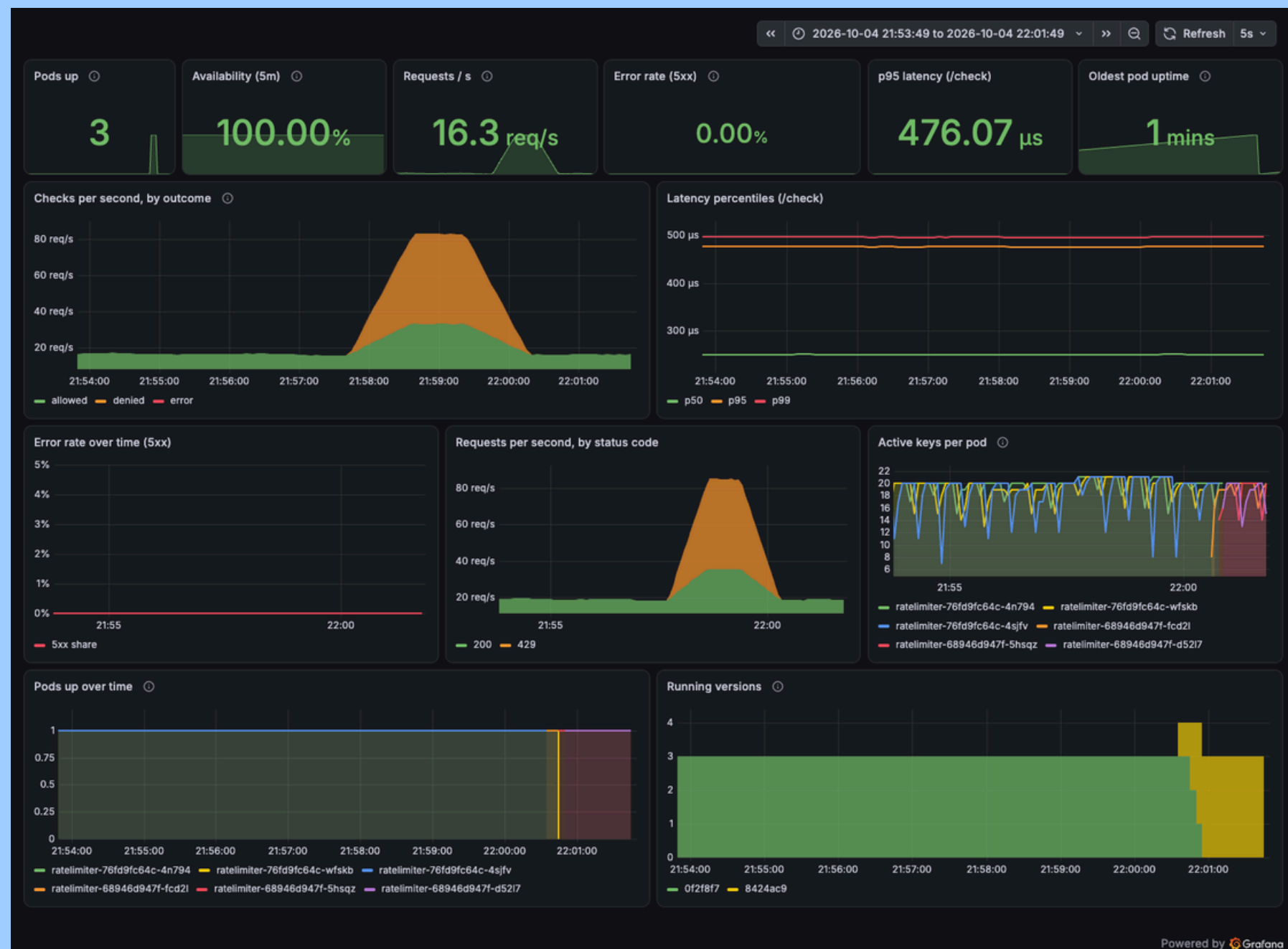
- Two-stage build: compile in one stage, ship only the binary.
- Static binary on a distroless base, so there's no shell or package manager.
- Runs as a non-root user.
- 17.7 MB multi-arch image (amd64 and arm64), published to GHCR by CI.

Kubernetes

- Namespace, ConfigMap, Deployment and Service.
- 3 replicas behind one stable Service address (port 80 to 8080).
- Rolling updates with maxUnavailable: 0, so capacity never drops.
- Readiness and liveness probes on /health.
- Hardened pods: read-only filesystem, all capabilities dropped.
- Graceful shutdown: stop taking traffic first, then finish in-flight requests.

The service is packaged as one small, immutable image, and Kubernetes keeps 3 copies running and replaces them safely when a new version ships.

Observability with Prometheus and Grafana



Metrics are built into the service itself, and Prometheus finds pods through the Kubernetes API, so monitoring keeps up automatically as pods are added, replaced or restarted.

Prometheus scrapes every pod's metrics and Grafana turns them into a live dashboard for uptime, latency and error rate.

Configuration Management with Ansible

```
Uttkarshs-Macbook-Air:ansible uttkarshruparel$ .venv/bin/ansible-playbook site.yml | grep -E 'PLAY RECAP|rl-node1'
ok: [rl-node1]
ok: [rl-node1]
ok: [rl-node1]
ok: [rl-node1]
ok: [rl-node1]
ok: [rl-node1]
ok: [rl-node1]
ok: [rl-node1]
ok: [rl-node1] => (item=/opt/ratelimiter)
ok: [rl-node1] => (item=/opt/ratelimiter/releases/local)
ok: [rl-node1] => (item=/etc/ratelimiter)
skipping: [rl-node1]
skipping: [rl-node1]
ok: [rl-node1]
ok: [rl-node1]
ok: [rl-node1]
ok: [rl-node1]
ok: [rl-node1]
ok: [rl-node1]
ok: [rl-node1]
ok: [rl-node1] =>
PLAY RECAP *****
rl-node1 : ok=17  changed=0  unreachable=0  failed=0  skipped=2  rescued=0  ignored=0
```

Ansible is an agentless automation tool that configures servers over SSH from a playbook, and here it takes a bare Ubuntu machine to a running, hardened ratelimiter service.

How it's designed

- **Idempotent:** the first run made 11 changes and the second made 0.
- **Self-healing:** it repairs configuration drift, such as an edited config file, a stopped service or a deleted file, in one run.
- **Versioned releases with instant rollback:** each version gets its own directory and a current symlink picks the live one.
- **Checksum-verified downloads:** a tampered release is refused and the running service is left untouched.

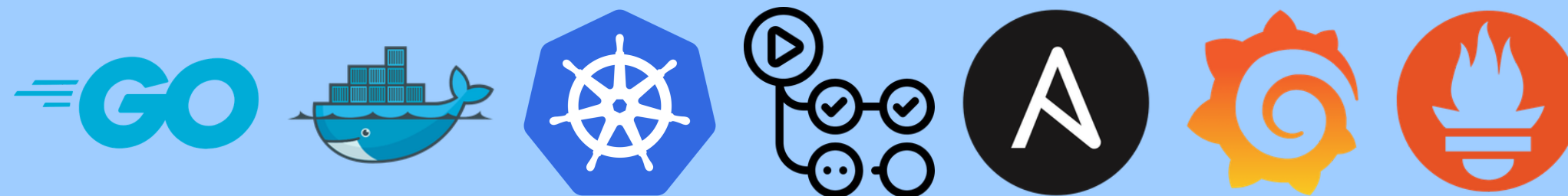
What it sets up

- **Packages:** installs the baseline tools the server needs.
- **Service account:** creates a locked-down ratelimiter user with no login shell.
- **Files:** manages the config file, the hardened systemd unit and the login banner.
- **Service:** installs the release binary, starts the service and checks /health afterwards.

Conclusion

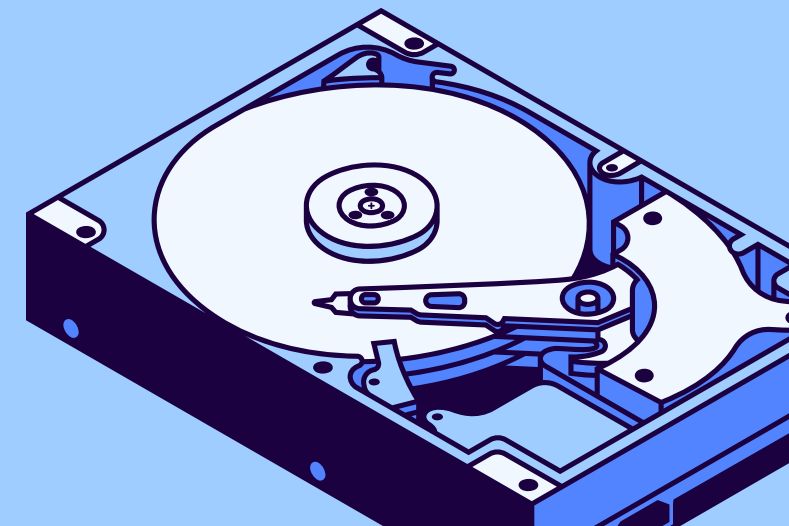
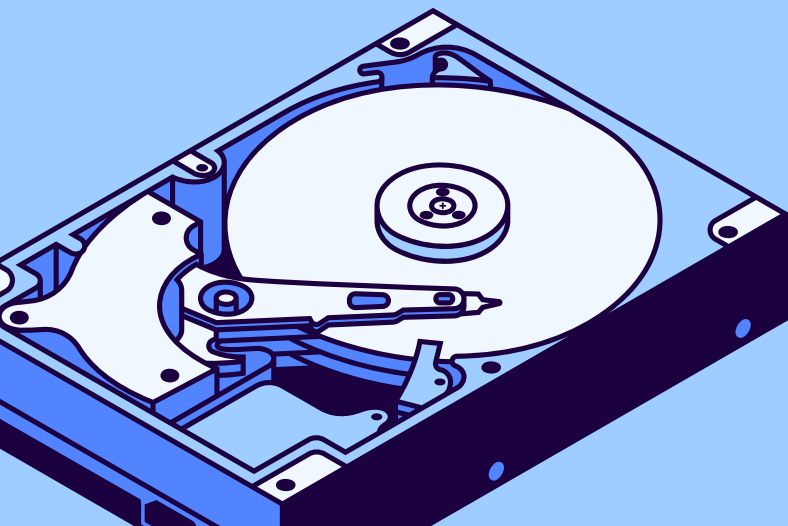
- One service, taken through the full lifecycle. A Go rate limiter went from commit to a monitored Kubernetes deployment, with Ansible covering the non-Kubernetes server.
- Automation removes guesswork. Every change is linted, tested and smoke-tested the same way, and the same image runs everywhere.
- Safe delivery is a design choice. Rolling updates, health probes and one-command rollbacks meant a bad release never took the service down.
- You can only fix what you can see. Built-in metrics and alerts caught every failure we caused on purpose.

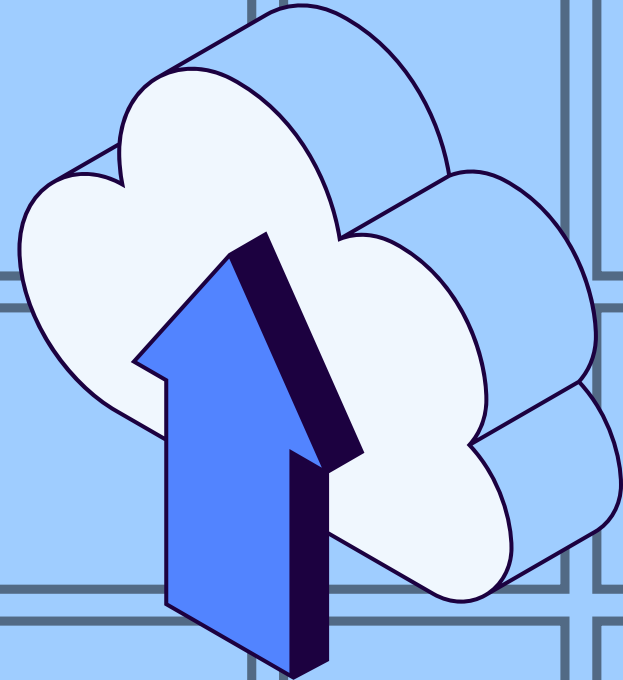
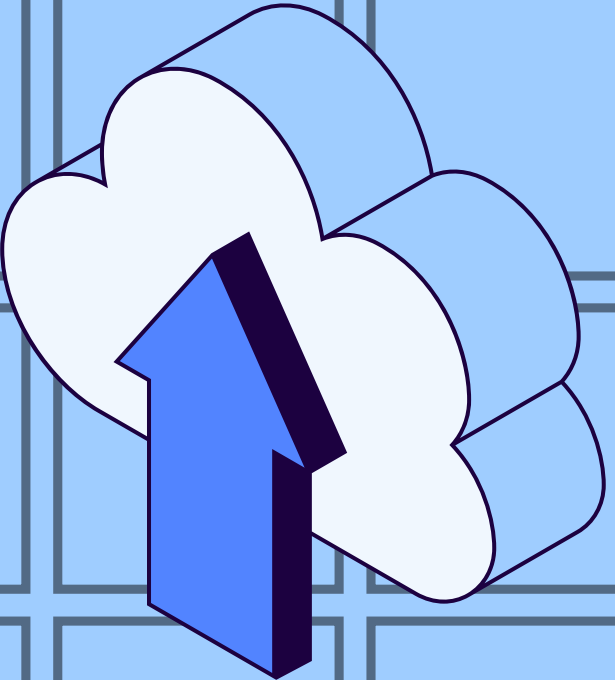
Built with



Source Code Repository:

<https://github.com/YHQZ1/local-rate-limiter-service>





Thank You!

